

DEVELOPER

02.09.2026

Contents

Data Vault 2.1 - Developer Guide	3
Inhaltsverzeichnis	3
Data Vault 2.0 Leitfaden	4
Grundprinzip	4
Entscheidungslogik	4
Hub	5
Satellite	5
Dependent Child (DC) Satellite	6
Multi-Active (MA) Satellite	7
Link	7
Link Satellite	8
Effectivity Satellite	8
Reference Table	8
PIT Table (Point-in-Time)	9
Information Mart	9
Häufige Fehlannahmen	9
Merksatz	10
Quick Reference	10
Häufigste Befehle	10
dbt Selektoren (Model Selection)	11
Wichtige Dateien	12
Parquet-Exploration Macros	12
External Table Macros (Optimiert)	13
Projektstruktur	14
Neues Attribut hinzufügen	15
Szenario	15
Schritt-für-Schritt	15
Wichtig	16
Neue Entity erstellen (Komplett)	16
Szenario	16
Übersicht der Schritte	17
Schritt 1: External Table definieren	17
Schritt 2: Staging View erstellen	18
Schritt 3: Hub erstellen	19

Schritt 4: Satellite erstellen	20
Schritt 5: Schema YAML erstellen (WICHTIG!)	21
Schritt 6: Deployment	24
Einzelne Objekte erstellen	24
5.1 Hub erstellen	24
5.2 Satellite erstellen	25
5.3 Link erstellen	26
5.4 Reference Table erstellen	27
5.5 PSA (Persistent Staging Area) erstellen	28
5.5 Effectivity Satellite erstellen	31
5.6 Dependent Child Satellite (DC Sat) erstellen	32
5.7 Multi-Active Satellite (MA Sat) erstellen	34
5.8 PIT Table erstellen	36
5.9 Current View erstellen (sat*_current_v)	37
Mart — Dimensionale Modellierung	37
Überblick	37
Surrogate Key Macro	37
Dimension Pflicht-Spalten	38
Beispiel: Dimension	38
Beispiel: Faktentabelle	38
Naming	39
Materialisierung	39
Schema YAML	39
ER-Diagramm	39
Agent	39
Mart View erstellen (Legacy Pattern)	40
Szenario	40
Static Tables (Persistierte Marts)	41
Übersicht	41
Wann Static Table vs. View?	42
Static Table erstellen	42
Index Macros	43
Deployment	44
Index verifizieren	44
CLI Command	44
Security in Mart-Models (RLS/CLS)	44
Regel 1: RLS-Schlüssel in Fakt-Models führen	45
Regel 2: Physische Mart-Tabelle Security Policy via Hook-Paar	45
Regel 3: Publizierte _v-Views rls_filter	45
Regel 4: PII-Spalten cls_mask, Tier-1 nie in den Mart	45
Referenz-Implementierungen	45
Häufige Fehler	45
Tests hinzufügen	46
Test-Typen	46
Tests in schema.yml	46
Tests ausführen	47
Deployment Workflow	47
GitHub Actions CI/CD Pipeline	47

Manuelles Deployment (Lokal)	48
Manuelles Prod-Deployment (ohne CI/CD)	49
Full Refresh (Schema-Änderungen)	49
Troubleshooting	49
Häufige Fehler	49
Debug-Befehle	50
Checklisten	50
Neue Entity Checkliste	50
Attribut hinzufügen Checkliste	50
Pre-Deployment Checkliste	51
CI/CD Troubleshooting	51
Weiterführende Dokumentation	51

Data Vault 2.1 - Developer Guide

Projekt: Virtual Data Vault 2.1 auf Azure **Version:** 2.0.0 **Stand:** 2025-12-27
Zielgruppe: Entwickler, Data Engineers

TEST RUN INDEX: 2

Inhaltsverzeichnis

1. Data Vault 2.0 Leitfaden
 2. Quick Reference
 3. Projektstruktur
 4. Neues Attribut hinzufügen
 5. Neue Entity erstellen (Komplett)
 6. Einzelne Objekte erstellen
 - Hub erstellen
 - Satellite erstellen
 - Link erstellen
 - Reference Table erstellen
 - PSA (Persistent Staging Area) erstellen
 - Effectivity Satellite erstellen
 - PIT Table erstellen
 7. Mart View erstellen
 8. Security in Mart-Models (RLS/CLS)
 9. Tests hinzufügen
 10. Deployment Workflow
 11. Troubleshooting
 12. Checklisten
-

Data Vault 2.0 Leitfaden

Wann, Warum und Wie werden Data Vault Objekte verwendet?

Grundprinzip

Data Vault trennt strikt zwischen:

Aspekt	Frage	Objekt
Identität	Was existiert?	Hub
Beziehung	Wie hängt etwas zusammen?	Link
Historie	Wie hat es sich über Zeit verändert?	Satellite

Ziele: Auditierbarkeit, Historisierung, Skalierbarkeit, Entkopplung von Quelle & Reporting

Entscheidungslogik

```
Gibt es einen stabilen Business Key? |
└ JA HUB |
|
Ändern sich Attribute über Zeit? |
└ JA SATELLITE |
|
Beschreibt es eine Beziehung zwischen Objekten? |
└ JA LINK |
└ Hat die Beziehung eigene Attribute? LINK SATELLITE |
|
Mehrere Werte ohne eigene Identität (z.B. Telefonnummern)? |
└ JA DEPENDENT CHILD SATELLITE |
|
Mehrere Werte gleichzeitig gültig (z.B. mehrere Rollen)? |
└ JA MULTI-ACTIVE SATELLITE |
|
Stabile Lookup-Werte (Länder, Status)? |
└ JA REFERENCE TABLE (kein Hub!) |
|
Performance-Problem bei Zeitabfragen? |
└ JA PIT TABLE |
```

Hub

Aspekt	Beschreibung
Zweck	Repräsentiert Business Keys , identifiziert fachliche Objekte eindeutig
Wann?	Es gibt einen stabilen, fachlichen Schlüssel
Beispiele	MitarbeiterNr, KundenNr, VertragsNr, Projekt-ID
Eigenschaften	Keine fachlichen Attribute, keine Historie, ein Eintrag pro BK
Schlüssel	Hash Key (technisch), Business Key bleibt erhalten

```
-- Struktur
hk_<entity> -- Hash Key (PK)
<business_key> -- Business Key (z.B. object_id)
dss_business_key -- Normierter Business Key (CONCAT_WS-basiert)
dss_load_date -- Ladezeitpunkt
dss_create_datetime -- Erstellungszeitpunkt
dss_record_source -- Quelle
```

Satellite

Aspekt	Beschreibung
Zweck	Trägt Attribute und Historie
Wann?	Attribute ändern sich über Zeit, Historisierung ist relevant
Best Practice	1 Thema = 1 Satellite, nach Änderungsfrequenz schneiden
Historisierung	Jede fachliche Änderung = neuer Datensatz

```
-- Struktur
hk_<entity> -- Hash Key (FK zum Hub)
dss_load_date -- Ladezeitpunkt (Teil des PK)
hd_<entity> -- Hash Diff (Änderungserkennung)
<attribute_1> -- Fachliche Attribute
<attribute_n>
dss_create_datetime -- Erstellungszeitpunkt
dss_is_current -- 'Y' = aktuell, 'N' = historisch
dss_end_date -- Gültigkeitsende (NULL = aktuell)
```

Varianten:

Typ	Wann verwenden?	Beispiel
Standard Satellite	Normale Attribute	sat_company
Dependent Child (DC)	Entity ohne eigene Identität, identifiziert über Parent-Beziehung + DCK	sat_contact_contractor_dc

Typ	Wann verwenden?	Beispiel
Multi-Active (MA)	Mehrere gleichzeitig gültige Werte	Mitarbeiter mit mehreren Rollen
Extension Satellite	Zusätzliche Attribute für Teilmenge	sat_company_client_ext (nur für Clients)

Dependent Child (DC) Satellite

Aspekt	Beschreibung
Zweck	Erfasst Entities ohne eigenen stabilen Business Key
Wann?	Entity existiert nur im Kontext eines Parent (z.B. Ansprechpartner zu Firma)
Identifikation	Parent-FK + Dependent Child Keys (DCK) bilden zusammen den logischen Schlüssel
Struktur	DC Satellite hängt am Link , nicht am Hub

Beispiel: Contact als Dependent Child von Contractor

```

hub_contractor
|
|   hk_contractor
|
link_contact_contractor
(hk_link = HASH(FK + DCK))
|
|   hk_link_contact_contractor
|
sat_contact_contractor_dc
(DCK: name, email im Payload)

```

Staging für DC Pattern:

```

-- Alle Hashes werden im Staging berechnet (automate_dv Best Practice)
hk_contractor -- FK Hash zum Parent Hub
hk_link_contact_contractor -- Link Hash = HASH(company_contractor ^^ name ^^ email)
hd_contact_contractor_dc -- Hashdiff für Änderungserkennung

```

Link Model (nur 1 FK für Pure DC):

```

src_pk: "hk_link_contact_contractor"
src_fk: "hk_contractor" # Nur Parent-FK, kein zweiter Hub
src_ldts: "dss_load_date"
src_source: "dss_record_source"

```

DC Satellite Model:

```
src_pk: "hk_link_contact_contractor" # Referenziert Link, nicht Hub
src_hashdiff:
  source_column: "hd_contact_contractor_dc"
  alias: "HASHDIFF"
src_payload:
  - "name" # DCK Column
  - "email1" # DCK Column
  - "phone" # Weitere Attribute
  - "..."
```

Multi-Active (MA) Satellite

Aspekt	Beschreibung
Zweck	Erfasst mehrere gleichzeitig gültige Werte
Wann?	Entity hat multiple aktive Zustände (z.B. mehrere Rollen)
Eigenschaften	Zusätzliches Attribut als Teil des PK zur Unterscheidung

Link

Aspekt	Beschreibung
Zweck	Modelliert Beziehungen zwischen Hubs
Wann?	n:m- oder 1:n-Beziehungen, Beziehung ist fachlich relevant
Beispiele	Mitarbeiter Projekt, Kunde Vertrag, Company Country
Eigenschaften	Enthält nur Schlüssel der beteiligten Hubs, keine Attribute

```
-- Struktur (Standard Link)
hk_link_<e1>_<e2> -- Link Hash Key (PK)
hk_<entity_1> -- FK zu Hub 1
hk_<entity_2> -- FK zu Hub 2
dss_load_date -- Ladezeitpunkt
dss_record_source -- Quelle
```

Link-Varianten:

Typ	FKs	Hash-Berechnung	Beispiel
Standard Link	2+ Hub-FKs	HASH(FK1 ^^ FK2)	link_company_country
DC Link (Pure)	1 Hub-FK	HASH(FK ^^ DCK1 ^^ DCK2)	link_contact_contractor

Typ	FKs	Hash-Berechnung	Beispiel
DC Link (Hybrid)	2 Hub-FKs + DC	$\text{HASH}(\text{FK1} \wedge \wedge \text{FK2} \wedge \wedge \text{DCK})$	Contact mit eigenem Hub + Parent

Link Satellite

Aspekt	Beschreibung
Zweck	Attribute, die die Beziehung beschreiben
Wann?	Attribute gelten für die Beziehung, nicht für das Objekt
Beispiele	Rolle eines Mitarbeiters im Projekt, Vertragsstatus

Effectivity Satellite

Aspekt	Beschreibung
Zweck	Trackt Gültigkeitszeiträume von Beziehungen
Wann?	Beziehungen können enden und wieder beginnen
Beispiele	Company-Country Zuordnung über Zeit

```
-- Struktur
hk_link_<e1>_<e2> -- FK zum Link (PK)
dss_start_date -- Beginn der Gültigkeit (PK)
dss_end_date -- Ende (NULL = aktiv)
dss_is_active -- 'Y' = aktiv, 'N' = beendet
```

Reference Table

Aspekt	Beschreibung
Zweck	Stabile, kleine Lookup-Tabellen
Wann?	Kaum Änderungen, keine Historisierung nötig
Beispiele	Länder, Währungen, Statuscodes, Rollen
Regeln	Nicht historisieren, nicht als Satellite, nicht übermodellieren

```
-- Beispiel: ref_role (als dbt Seed)
role_code -- PK (CLIENT, CONTRACTOR, SUPPLIER)
role_name -- Anzeigename
role_description -- Beschreibung
```


PIT Table (Point-in-Time)

Aspekt	Beschreibung
Zweck Wann?	Performance-Optimierung für "As-of"-Abfragen Viele Satellites, komplexe zeitbezogene Joins, BI-Performance kritisch
Eigenschaften Wichtig	Rein technisch, keine fachlichen Attribute Kein Pflichtbestandteil – nur bei Bedarf einsetzen!

```
-- Struktur  
hk_<entity> -- FK zum Hub  
snapshot_date -- Zeitpunkt  
hk_sat_<name> -- Verweis auf gültigen Satellite-Zustand  
dss_load_date_sat -- Load Date des referenzierten Satellites
```

Information Mart

Aspekt	Beschreibung
Zweck Eigenschaften	Konsum-Schicht für BI & Analytics Dimensions- und Faktenmodelle, abgeleitet aus Raw/Business Vault
Inhalte Wichtig	dim_date, dim_kunde, fakt_rechnung Keine unabhängige Modellierung, keine zusätzliche Historisierung

Häufige Fehlannahmen

Falsch	Richtig
Hubs sind historisiert	Hubs haben nur Ladezeitpunkt, keine fachliche Historie
Alles braucht einen Hub	Lookup-Werte Reference Table
PIT ist Pflicht	PIT nur bei Performance-Bedarf
Referenztabellen in Satellites	Reference Tables sind eigenständig
Information Mart ist eigenes DWH	Mart ist nur View-Schicht auf Vault

Merksatz

Hubs identifizieren. Satellites historisieren. Links verbinden. Dependent Children ergänzen. Multi-Active gilt parallel. PIT beschleunigt. Information Marts erklären.

Quick Reference

Häufigste Befehle

Umgebung aktivieren

```
cd ~/projects/datavault-dbt && source .venv/bin/activate
```

Verbindung testen

```
dbt debug
```

Models bauen

```
dbt run # Alle Models
```

```
dbt run --select raw_vault.jira.hub_company # Einzelnes Model (empfohlen)
```

```
dbt run --select +raw_vault.jira.sat_company+ # Model mit Abhängigkeiten
```

```
dbt run --full-refresh # Alles neu bauen
```

External Tables erstellen / aktualisieren

Namenskonvention: ext_<concept>_<entity> (z.B. ext_jira_project, ext_jira_company)

Option 1: ALLE External Tables (Standard)

```
dbt run-operation stage_external_sources
```

oder einzelne Tabelle (Format: staging.<table_name>)

```
dbt run-operation stage_external_sources --args 'select: staging.ext_jira_project'
```

Full Refresh (DROP + CREATE, bei Schema-Änderungen)

```
dbt run-operation stage_external_sources --vars 'ext_full_refresh: true'
```

Full Refresh für einzelne Tabelle

```
dbt run-operation stage_external_sources --args 'select: staging.ext_jira_project'
```

Source view erstellen

```
dbt run --select jira_project
```

Option 2: EINZELNE neue Tabelle (optimiert)

Nur die neue ext_jira_project erstellen (schneller)

```
dbt run-operation create_external_table \
  --args 'table_name: ext_jira_project'
```

Option 3: Explorieren (ohne zu erstellen)

Parquet-Dateien in ADLS erkunden

```
dbt run-operation list_parquet_files --args '{"folder_path": "jira/sql"}'
```

```
dbt run-operation get_parquet_schema --args '{"folder_path": "jira/sql", "file_name": "jira_project.parquet"}'
```

```
dbt run-operation get_parquet_data --args '{"folder_path": "jira/sql", "file_name": "jira_project.parquet"}'
```

Source view erstellen

```
dbt run --select adventureworks_customer
```

Tests

```
dbt test # Alle Tests
```

```
dbt test --select hub_company # Tests für ein Model
```

Seeds (Reference Data)

```
dbt seed # Alle Seeds laden
```

Kompilieren (SQL anzeigen ohne Ausführung)

```
dbt compile --select model_name
```

```
cat target/compiled/datavault/models/path/to/model.sql
```

dbt Selektoren (Model Selection)

Wichtig: Verwende immer den **vollständigen Pfad**, da Model-Namen (z.B. hub_contacts) in mehreren Concepts existieren können!

Vermeiden - wählt ALLE hub_company in allen Concepts

```
dbt run --select hub_company
```

Empfohlen - spezifischer Pfad

```
dbt run --select raw_vault.jira.hub_company
```

Pfad-Pattern für einzelne Datei

```
dbt run --select path:models/raw_vault/jira/hubs/hub_company.sql
```

Selektor-Syntax:

Selektor	Beschreibung	Beispiel
raw_vault.jira.hub_company	Pfad-basiert (Ordnerstruktur)	Empfohlen für einzelne Models
raw_vault.jira	Alle Models eines Concepts	Für Concept-Deployment
staging.jira_* +model_name	Wildcard-Pattern Model inkl. Upstream-Dependencies	Alle Jira-Staging Views +hub_company baut erst Staging
model_name+ +model_name+	Model inkl. Downstream-Dependents	hub_company+ baut auch Satellites
tag:static	Nach Tag	Vollständige Dependency-Chain Alle statischen Tabellen

Pfad-Mapping:

```
models/
├── staging/ staging.*
├── raw_vault/
│   ├── jira/ raw_vault.jira.*
│   │   ├── hubs/ raw_vault.jira.hub_*
│   │   └── satellites/ raw_vault.jira.sat_*
│   └── adventureworks/ raw_vault.adventureworks.*
├── business_vault/ business_vault.*
├── mart/
│   └── project/ mart.project.*
```

Kombinierte Selektoren:

Staging + Hub + Satellite für eine Entity

```
dbt run --select raw_vault.jira.hub_company raw_vault.jira.sat_company
```

Oder mit Upstream-Dependencies (baut auch Staging automatisch)

```
dbt run --select +raw_vault.jira.hub_company +raw_vault.jira.sat_company
```

Alle Jira Raw Vault Models

```
dbt run --select raw_vault.jira
```

Nur Hubs eines Concepts

```
dbt run --select raw_vault.jira.hub_*
```

Wichtige Dateien

Datei	Zweck	Link
dbt_project.yml	Projektkonfiguration	öffnen
models/staging/sources.yml	External Tables Definition (ext_<concept>_<entity>)	öffnen
models/schema.yml	Tests & Dokumentation	öffnen
macros/generate_schema_names.sql	Schema-Naming	öffnen
macros/satellite_current_flag.sql	Current Flag Macro	öffnen
macros/ghost_records.sql	Ghost Records	öffnen

Parquet-Exploration Macros

Für die Analyse von Parquet-Dateien in ADLS Gen2 stehen drei Macros zur Verfügung:

Macro	Zweck	Befehl
list_parquet_files	Alle Dateien in einem ADLS-Ordner auflisten	dbt run-operation list_parquet_files --args '{"folder_path": "jira/sql"}'

Macro	Zweck	Befehl
get_parquet_schema	Schema einer Datei als YAML für sources.yml ausgeben	dbt run-operation get_parquet_schema --args '{"folder_path": "jira/sql", "file_name": "Plat- form.Api_Project.parquet"}'
get_parquet_data	Beispieldaten einer Datei anzeigen	dbt run-operation get_parquet_data --args '{"folder_path": "jira/sql", "file_name": "Plat- form.Api_Project.parquet", "limit": 5}'

External Table Macros (Optimiert)

Neue Macros für **selektive** External Table Erstellung (nicht alle auf einmal):

Macro	Zweck	Befehl
create_external_table	Erstellt nur EINE neue Tabelle basierend auf sources.yml	dbt run-operation create_external_table --args '{"table_name": "ext_jira_project"}'
stage_external_sources	Standard dbt-external-tables: Erstellt ALLE Tabellen (idempotent)	dbt run-operation stage_external_sources

Best Practice: - **Neue Tabelle?** create_external_table (schneller, nur eine) - **Alle Tabellen?** stage_external_sources (vollständig, sicher)

Typischer Workflow für neue Datenquelle:

1. *Verfügbare Dateien anzeigen*

```
dbt run-operation list_parquet_files --args '{"folder_path": "neue_quelle/ordner"}
```

2. *Schema einer Datei als YAML generieren (direkt in sources.yml kopierbar)*

```
dbt run-operation get_parquet_schema --args '{"folder_path": "neue_quelle/ordner",
```

3. *Optional: Beispieldaten prüfen*

```
dbt run-operation get_parquet_data --args '{"folder_path": "neue_quelle/ordner", "
```

Voraussetzungen: - External Data Source StageFileSystem muss in der Datenbank existieren - ADLS Gen2 Container muss über PolyBase/OPENROWSET erreichbar sein

Projektstruktur

```
datavault-dbt/
├── dbt_project.yml # Projektkonfiguration
├── packages.yml # Package-Abhängigkeiten
├── profiles.yml # In ~/.dbt/ (nicht im Repo!)
├── macros/ # Wiederverwendbare Macros
│   ├── generate_schema_name.sql
│   ├── satellite_current_flag.sql
│   └── ghost_records.sql
├── seeds/ # Reference Data (CSV)
│   └── ref_role.csv
├── models/
│   ├── schema.yml # Tests & Dokumentation
│   ├── staging/ # Schema: stg
│   │   ├── sources.yml # External Table Definitionen
│   │   ├── jira_company.sql # Staging Views
│   │   └── jira_country.sql
│   ├── raw_vault/ # Raw Vault Layer
│   │   ├── _common/ # Schema: vault (source-übergreifend)
│   │   │   ├── hubs/
│   │   │   ├── satellites/
│   │   │   └── links/
│   │   ├── jira/ # Schema: vault_jira
│   │   │   ├── hubs/
│   │   │   │   ├── hub_company.sql
│   │   │   │   └── hub_country.sql
│   │   │   ├── satellites/
│   │   │   │   ├── sat_company.sql
│   │   │   │   └── eff_sat_company_country.sql
│   │   │   └── links/
│   │   │       └── link_company_country.sql
│   │   ├── adventureworks/ # Schema: vault_adventureworks
│   │   │   ├── hubs/
│   │   │   │   └── hub_customer.sql
│   │   │   ├── satellites/
│   │   │   │   └── sat_customer.sql
│   │   │   └── links/
│   ├── business_vault/ # Schema: vault (PITs, Bridges)
│   │   └── pit_company.sql
│   └── mart/ # Mart Layer (für BI)
```

```

├── _common/ # Schema: mart (geteilte Dimensionen)
│   ├── dim_date.sql
│   ├── dim_kunde.sql
│   └── project/ # Schema: mart_project
│       ├── dim_projekt.sql
│       └── fakt_projekt.sql
├── docs/ # Dokumentation
│   ├── SYSTEM.md
│   ├── USER.md
│   ├── DEVELOPER.md # Diese Datei
│   └── MODEL_ARCHITECTURE.md
├── target/ # Kompilierte Artefakte
└── compiled/ # Generiertes SQL

```

Neues Attribut hinzufügen

Szenario

Ein bestehendes Attribut soll zum Satellite hinzugefügt werden (z.B. tax_number zu sat_company).

Schritt-für-Schritt

Schritt 1: External Table erweitern **Datei:** models/staging/sources.yml

```

# Finde die External Table und füge die Spalte hinzu
- name: ext_jira_company
  columns:
  # ... bestehende Spalten ...
  - name: tax_number # NEU
  data_type: NVARCHAR(50) # Datentyp

```

Schritt 2: Staging View erweitern **Datei:** models/staging/jira_company.sql

Staging Views verwenden das automate_dv.stage() YAML Metadata Pattern. Füge die neue Spalte zum hashed_columns Block hinzu:

```

# In der yaml_metadata Sektion:
hashed_columns:
  hk_company: "object_id"
  hd_company:
    is_hashdiff: true
  columns:
  - "name"
  - "street"

```

```
# ... bestehende ...
```

```
- "tax_number" # NEU (falls Änderungen getrackt werden sollen)
```

Hinweis: Hashdiff-Spalten werden automatisch alphabetisch sortiert durch automate_dv. Die Spalte muss an der richtigen alphabetischen Position eingefügt werden.

Schritt 3: Satellite erweitern **Datei:** models/raw_vault/satellites/sat_company.sql

```
WITH source_data AS (  
  SELECT  
    hk_company,  
    hd_company,  
    -- ... bestehende Spalten ...  
    tax_number, -- NEU  
    dss_load_date,  
    dss_record_source  
  FROM {{ ref('jira_company') }}  
  WHERE hk_company IS NOT NULL  
)  
-- ... Rest bleibt gleich ...
```

Schritt 4: Deployment

```
# External Table aktualisieren
```

```
dbt run-operation stage_external_sources
```

```
# Satellite neu bauen (full-refresh wegen Schemaänderung!)
```

```
dbt run --full-refresh --select jira_company sat_company
```

```
# Tests ausführen
```

```
dbt test --select sat_company
```

Wichtig

- Bei **Schema-Änderungen** immer --full-refresh verwenden
- Hash Diff nur erweitern wenn Änderungen getrackt werden sollen
- Nach Änderung: Tests ausführen!

Neue Entity erstellen (Komplett)

Szenario

Eine komplett neue Entity soll ins Data Vault (z.B. product aus einer neuen Quellta-
belle).

Übersicht der Schritte

```
1. External Table 2. Staging View 3. Hub |
|
sources.yml jira_product.sql hub_product.sql |
|
4. Satellite 5. Link |
sat_product.sql link_*.sql |
|
6. Tests & Deploy |
```

Schritt 1: External Table definieren

Datei: models/staging/sources.yml

```
sources:
  - name: staging
    database: "{{ target.database }}"
    schema: stg
    tables:
      # ... bestehende Tabellen ...

      # =====
      # NEU: Product
      # =====
      - name: ext_jira_product
        external:
          location: "jira/postgres/public.wp_product.parquet"
          file_format: ParquetFormat
          columns:
            - name: object_id
              data_type: BIGINT
            - name: name
              data_type: NVARCHAR(255)
            - name: description
              data_type: NVARCHAR(MAX)
            - name: price
              data_type: DECIMAL(18,2)
            - name: category_id
              data_type: BIGINT
            - name: dss_record_source
              data_type: NVARCHAR(100)
            - name: dss_load_date
              data_type: DATETIME2
```

```
- name: dss_run_id
data_type: NVARCHAR(100)
```

Schritt 2: Staging View erstellen

Neue Datei: models/staging/jira_product.sql

Staging Views verwenden das **automate_dv.stage() YAML Metadata Pattern**. Alle Hash Keys und Hash Diffs werden automatisch von automate_dv berechnet (mit Custom Overrides in macros/hash_override.sql).

```
/*
 * Staging Model: jira_product
 *
 * Source: ext_jira_product (Product-Daten)
 * Business Key: object_id
 * Hash Key: hk_product
 * Payload: 4 Spalten – Product-Attribute
 *
 * Uses automate_dv.stage() macro for standardized staging.
 */

{% set yaml_metadata %}
source_model:
  staging: "ext_jira_product"

derived_columns:
  dss_record_source: "!jira"
  dss_load_date: "COALESCE(TRY_CAST(dss_load_date AS DATETIME2), GETDATE())"
  dss_create_datetime: "GETDATE()"
  dss_business_key: "CONCAT_WS('||', 'default', 'default', ISNULL(LTRIM(RTRIM(CAST(

hashed_columns:
  hk_product: "object_id"
  hk_category: "category_id"
  hk_link_product_category:
    - "object_id"
    - "category_id"
  hd_product:
  is_hashdiff: true
  columns:
    - "category_id"
    - "description"
    - "name"
    - "price"
{% endset %}

{% set metadata_dict = fromyaml(yaml_metadata) %}
```

```
{{ automate_dv.stage(include_source_columns=true,
source_model=metadata_dict['source_model'],
derived_columns=metadata_dict['derived_columns'],
hashed_columns=metadata_dict['hashed_columns']) }}
```

EWB-spezifische Referenz-Beispiele: - Single BK + Reserved Keyword: models/staging/ewb_lohn_len_main.sql - Composite BK: models/staging/ewb_proj_nsa_main.sql - Multiple Reserved Keywords: models/staging/ewb_fibu_fhe_main.sql

Schritt 3: Hub erstellen

Neue Datei: models/raw_vault/hubs/hub_product.sql

```
/*
 * Hub: hub_product
 * Schema: vault
 *
 * Speichert eindeutige Product Business Keys.
 * Insert-Only: Neue Products werden hinzugefügt, nie gelöscht.
 */
```

```
{{ config(
    materialized='incremental',
    unique_key='hk_product',
    as_columnstore=false
) }}
```

```
WITH source_data AS (
    SELECT DISTINCT
        hk_product,
        object_id,
        dss_load_date,
        dss_record_source
    FROM {{ ref('jira_product') }}
    WHERE hk_product IS NOT NULL
),
```

```
{% if is_incremental() %}
existing_hubs AS {
    SELECT hk_product FROM {{ this }}
},
{% endif %}
```

```
new_records AS (
    SELECT
        src.hk_product,
        src.object_id,
        src.dss_load_date,
```

```

src.dss_record_source
FROM source_data src
{% if is_incremental() %}
WHERE NOT EXISTS (
SELECT 1 FROM existing_hubs eh
WHERE eh.hk_product = src.hk_product
)
{% endif %}
)

```

```
SELECT * FROM new_records
```

Schritt 4: Satellite erstellen

Neue Datei: models/raw_vault/satellites/sat_product.sql

```

/*
 * Satellite: sat_product
 * Schema: vault
 *
 * Speichert Product-Attribute mit vollständiger Historie.
 * dss_is_current: 'Y' für aktuellen Eintrag
 * dss_end_date: Ende der Gültigkeit
 */

{{ config(
    materialized='incremental',
    unique_key='hk_product',
    as_columnstore=false,
    post_hook=[
        "{{ update_satellite_current_flag(this, 'hk_product') }}"
    ]
) }}

```

```

WITH source_data AS (
    SELECT
        hk_product,
        hd_product,
        dss_load_date,
        dss_record_source,
        -- Payload
        name,
        description,
        price,
        category_id
    FROM {{ ref('jira_product') }}
    WHERE hk_product IS NOT NULL
),

```

```

{% if is_incremental() %}
existing_sats AS (
  SELECT
    hk_product,
    hd_product
  FROM {{ this }}
),
{% endif %}

new_records AS (
  SELECT
    src.hk_product,
    src.hd_product,
    src.dss_load_date,
    src.dss_record_source,
    src.name,
    src.description,
    src.price,
    src.category_id
  FROM source_data src
  {% if is_incremental() %}
  WHERE NOT EXISTS (
    SELECT 1 FROM existing_sats es
    WHERE es.hk_product = src.hk_product
    AND es.hd_product = src.hd_product
  )
  {% endif %}
)

SELECT
  *,
  'Y' AS dss_is_current,
  CAST(NULL AS DATETIME2) AS dss_end_date
FROM new_records

```

Schritt 5: Schema YAML erstellen (WICHTIG!)

Jedes Model MUSS in einer Schema YAML-Datei dokumentiert werden!

Die VS Code Extension und dbt-Dokumentation verwenden diese Dateien für Spalten-Metadaten.

Datei-Namenskonvention

Layer	Datei	Speicherort
Staging	_staging__models.yml	models/staging/

Layer	Datei	Speicherort
Raw Vault	_<concept>__models.yml	models/raw_vault/<concept>/
Business Vault	_business_vault__models.yml	models/business_vault/
Mart	_<concept>__models.yml	models/mart/<concept>/

Vorlage Datei: models/raw_vault/<concept>/_<concept>__models.yml

version: 2

models:

```
# =====
# Staging: Product
# =====
- name: <concept>_product
description: Staging view for product with hash calculations
columns:
- name: hk_product
description: Hash Key (Primary Key)
data_type: char(64)
tests:
- not_null
- name: object_id
description: Business Key from source
data_type: bigint
tests:
- not_null
- name: name
description: Product name
data_type: nvarchar(4000)
- name: dss_load_date
description: Load timestamp
data_type: datetime2(7)
tests:
- not_null
- name: dss_record_source
description: Data source identifier
data_type: varchar(100)
tests:
- not_null

# =====
# Hub: Product
# =====
- name: hub_product
description: Hub containing unique product business keys
columns:
- name: hk_product
```

```

description: Hash Key (Primary Key)
data_type: char(64)
tests:
- unique
- not_null
- name: object_id
description: Business Key
data_type: bigint
tests:
- not_null
- name: dss_load_date
description: First load timestamp
data_type: datetime2(7)
tests:
- not_null
- name: dss_record_source
description: Data source
data_type: varchar(100)
tests:
- not_null

# =====
# Satellite: Product
# =====
- name: sat_product
description: Satellite with product descriptive attributes
columns:
- name: hk_product
description: Hash Key (Foreign Key to Hub)
data_type: char(64)
tests:
- not_null
- relationships:
to: ref('hub_product')
field: hk_product
- name: hd_product
description: Hash Diff for change detection
data_type: char(64)
tests:
- not_null
- name: name
description: Product name
data_type: nvarchar(4000)
- name: dss_load_date
description: Load timestamp
data_type: datetime2(7)
- name: dss_record_source
description: Data source

```

```
data_type: varchar(100)
```

Spalten aus Datenbank generieren Mit VS Code Copilot und MSSQL MCP können Spaltendefinitionen aus bestehenden Views generiert werden:

```
SELECT c.name, t.name AS data_type, c.max_length, c.precision, c.scale, c.is_nullable
FROM sys.views v
JOIN sys.columns c ON v.object_id = c.object_id
JOIN sys.types t ON c.user_type_id = t.user_type_id
WHERE SCHEMA_NAME(v.schema_id) = 'stg' AND v.name = '<view_name>'
ORDER BY c.column_id;
```

Schritt 6: Deployment

1. External Table erstellen

```
dbt run-operation stage_external_sources
```

oder einzelne Tabelle

```
dbt run-operation stage_external_sources --args 'select: staging.ext_jira_product'
```

2. Alle neuen Models bauen (mit Upstream-Dependencies)

```
dbt run --select +raw_vault.jira.hub_product +raw_vault.jira.sat_product
```

3. Tests ausführen

```
dbt test --select raw_vault.jira.hub_product raw_vault.jira.sat_product
```

4. Ghost Records hinzufügen (optional)

Macro in ghost_records.sql erweitern

Einzelne Objekte erstellen

5.1 Hub erstellen

Vorlage: models/raw_vault/hubs/hub_company.sql

Minimales Template:

```
{{ config(
  materialized='incremental',
  unique_key='hk_<entity>',
  as_columnstore=false
) }}
```

```
WITH source_data AS (
  SELECT DISTINCT
    hk_<entity>,
    <business_key_columns>,
    dss_load_date,
```



```

dss_record_source
FROM {{ ref('<concept>_<entity>') }}
WHERE hk_<entity> IS NOT NULL
),

{% if is_incremental() %}
existing_hubs AS (
    SELECT hk_<entity> FROM {{ this }}
),
{% endif %}

new_records AS (
    SELECT *
    FROM source_data src
    {% if is_incremental() %}
    WHERE NOT EXISTS (
        SELECT 1 FROM existing_hubs eh
        WHERE eh.hk_<entity> = src.hk_<entity>
    )
    {% endif %}
)

```

```
SELECT * FROM new_records
```

Ersetzen: - <entity> Name der Entity (z.B. product) - <business_key_columns> Spalten des Business Keys

5.2 Satellite erstellen

Vorlage: models/raw_vault/satellites/sat_company.sql

Minimales Template:

```

{{ config(
    materialized='incremental',
    unique_key='hk_<entity>',
    as_columnstore=false,
    post_hook=[
        "{{ update_satellite_current_flag(this, 'hk_<entity>') }}"
    ]
) }}

```

```

WITH source_data AS (
    SELECT
        hk_<entity>,
        hd_<entity>,
        dss_load_date,
        dss_record_source,

```

```

-- Payload Spalten hier
<payload_columns>
FROM {{ ref('<concept>_<entity>') }}
WHERE hk_<entity> IS NOT NULL
),

{% if is_incremental() %}
existing_sats AS (
    SELECT hk_<entity>, hd_<entity> FROM {{ this }}
),
{% endif %}

new_records AS (
    SELECT *
    FROM source_data src
    {% if is_incremental() %}
    WHERE NOT EXISTS (
        SELECT 1 FROM existing_sats es
        WHERE es.hk_<entity> = src.hk_<entity>
        AND es.hd_<entity> = src.hd_<entity>
    )
    {% endif %}
)

SELECT
    *,
    'Y' AS dss_is_current,
    CAST(NULL AS DATETIME2) AS dss_end_date
FROM new_records

```

5.3 Link erstellen

Vorlage: models/raw_vault/links/link_company_role.sql

Minimales Template:

```

{{ config(
    materialized='incremental',
    unique_key='hk_link_<entity1>_<entity2>',
    as_columnstore=false
)}}

WITH source_data AS (
    SELECT DISTINCT
    hk_link_<entity1>_<entity2>,
    hk_<entity1>,
    hk_<entity2>,

```

```

dss_load_date,
dss_record_source
FROM {{ ref('<concept>_<source>') }}
WHERE hk_<entity1> IS NOT NULL
AND hk_<entity2> IS NOT NULL
),

{% if is_incremental() %}
existing_links AS (
    SELECT hk_link_<entity1>_<entity2> FROM {{ this }}
),
{% endif %}

new_records AS (
    SELECT *
    FROM source_data src
    {% if is_incremental() %}
    WHERE NOT EXISTS (
        SELECT 1 FROM existing_links el
        WHERE el.hk_link_<entity1>_<entity2> = src.hk_link_<entity1>_<entity2>
    )
    {% endif %}
)

SELECT * FROM new_records

```

Wichtig: Der Link Hash Key muss im Staging berechnet werden:

```

-- In <concept>_<source>.sql
CONVERT(CHAR(64), HASHBYTES('SHA2_256',
    CONCAT(
        ISNULL(LTRIM(RTRIM(CAST(<entity1_bk> AS NVARCHAR(MAX)))), '-1'),
        '^',
        ISNULL(LTRIM(RTRIM(CAST(<entity2_bk> AS NVARCHAR(MAX)))), '-1')
    )
), 2) AS hk_link_<entity1>_<entity2>

```

5.4 Reference Table erstellen

Vorlage: seeds/ref_role.csv

Schritt 1: CSV-Datei erstellen

```

role_code,role_name,role_description
CLIENT,Kunde,Unternehmen das Dienstleistungen bezieht
CONTRACTOR,Auftragnehmer,Unternehmen das Aufträge ausführt
SUPPLIER,Lieferant,Unternehmen das Waren liefert

```

Speichern als: seeds/ref_<name>.csv

Schritt 2: Konfiguration in dbt_project.yml

Datei: dbt_project.yml

```
seeds:
  datavault:
    +schema: vault
    ref_<name>:
    +column_types:
    <column>: VARCHAR(50)
```

Schritt 3: Deployment

```
dbt seed --select ref_<name>
```

5.5 PSA (Persistent Staging Area) erstellen

Zweck: PSA cached Daten aus External Tables (PolyBase/OPENROWSET) in einer inkrementellen SQL-Tabelle, um wiederholte teure Zugriffe auf ADLS zu vermeiden.

Wann PSA verwenden?

Szenario	Empfehlung
Kleine Parquet-Dateien, schnelle Abfragen	Keine PSA nötig
Große Datenmengen, häufige dbt runs	PSA verwenden
Merge/Upsert-Logik erforderlich	PSA verwenden
Inkrementelle Verarbeitung	PSA verwenden

Datenfluss mit PSA

ext_<concept>_<entity> (External Table - PolyBase)

psa_<concept>_<entity> (PSA - Incremental dbt Table) NEUER LAYER

<concept>_<entity> (Staging View - Hash-Berechnungen)

Hub/Satellite/Link (Raw Vault)

Wichtig: Die Staging View muss angepasst werden, um die PSA statt der External Table zu referenzieren!

Schritt 1: PSA-Tabelle erstellen **Neue Datei:** models/staging/psa_<concept>_<entity>.sql

```
/*
 * Persistent Staging Area: psa_<concept>_<entity>
 */
```

```

* Source: ext_<concept>_<entity>
* Strategy: merge
* Unique Key: <business_key>
* Incremental Column: dss_load_date
*
* Purpose: Persists external table data to avoid repeated OPENROWSET calls.
* Staging views (hash calculation) should reference this PSA table.
*/

```

```

{{ config(
    materialized='incremental',
    incremental_strategy='merge',
    unique_key='<business_key>',
    as_columnstore=false
) }}

```

SELECT

```

<business_key>,
<alle_spalten>,
dss_record_source,
dss_load_date,
dss_run_id,
dss_stage_timestamp,
dss_source_file_name

```

```

FROM {{ source('staging', 'ext_<concept>_<entity>') }}

```

```

{% if is_incremental() %}

```

```

WHERE dss_load_date > (SELECT COALESCE(MAX(dss_load_date), '1900-01-01') FROM {{ th

```

```

{% endif %}

```

Incremental Strategy Optionen:

Strategy	Beschreibung	Unique Key erforderlich?
merge	Update bei Match, Insert bei neuem Record	Ja
append	Nur neue Records einfügen (kein Update)	Nein

Schritt 2: sources.yml aktualisieren Datei: models/staging/sources.yml

Füge einen Eintrag für die PSA-Tabelle hinzu mit meta.psa: true:

```

# PSA für Entity
- name: <concept>_<entity> # Ohne ext_Prefix
description: Persistent Staging Area for ext_<concept>_<entity>
meta:
  psa: true # Markiert als PSA für Tree View
source_external_table: ext_<concept>_<entity>
columns:

```

```
- name: <business_key>
data_type: BIGINT
# ... weitere Spalten
```

Schritt 3: Staging View anpassen (KRITISCH!) Die bestehende Staging View muss angepasst werden, um die PSA zu referenzieren!

Datei: models/staging/<concept>_<entity>.sql

-- VORHER:

```
WITH source AS (
  SELECT * FROM {{ source('staging', 'ext_<concept>_<entity>') }}
),
```

-- NACHHER:

```
WITH source AS (
  SELECT * FROM {{ ref('psa_<concept>_<entity>') }}
),
```

Warum diese Änderung? - Die Hash-Berechnungen (hk_, hd_) erfolgen weiterhin in der Staging View - Aber die Datenquelle ist jetzt die PSA-Tabelle statt der External Table
- Das reduziert teure OPENROWSET-Aufrufe bei jedem dbt run

Schritt 4: Schema YAML dokumentieren **Datei:** models/staging/_staging__models.yml

```
- name: psa_<concept>_<entity>
description: "Persistent Staging Area for <entity>. Caches data from ext_<concept>_<entity>"
columns:
  - name: <business_key>
    description: Business Key / Unique Identifier
    data_type: bigint
    tests:
      - not_null
# ... weitere Spalten dokumentieren
```

Schritt 5: Deployment

```
# 1. PSA erstellen (lädt alle Daten)
dbt run --select psa_<concept>_<entity>
```

```
# 2. Staging View anpassen (s.o.)
```

```
# 3. Alle abhängigen Models neu bauen
dbt run --select +<concept>_<entity>+
```

VS Code Extension Die VS Code Extension bietet den Befehl "Create PSA Table" im Kontextmenü der External Tables. Dieser erstellt automatisch: - Die PSA SQL-Datei - Den sources.yml Eintrag - Die Schema YAML Dokumentation

Die Staging View muss manuell angepasst werden! Die Extension ändert nur die PSA, nicht die referenzierende Staging View.

5.5 Effectivity Satellite erstellen

Vorlage: models/raw_vault/satellites/eff_sat_company_country.sql

Für Links die **Gültigkeitszeiträume** haben (z.B. "Firma war von 2020-2023 in Deutschland").

```
{{ config(
    materialized='incremental',
    unique_key=['hk_<hub>', 'dss_start_date'],
    as_columnstore=false,
    post_hook=[
        "{{ update_effectivity_end_dates() }}"
    ]
) }}
```

```
WITH source_data AS (
    SELECT
        hk_<hub>,
        hk_<related_hub>,
        hk_link_<entity1>_<entity2>,
        dss_load_date AS dss_start_date,
        dss_record_source
    FROM {{ ref('<concept>_<source>') }}
),

{% if is_incremental() %}
existing AS (
    SELECT hk_<hub>, dss_start_date FROM {{ this }}
),
{% endif %}

new_records AS (
    SELECT *
    FROM source_data src
    {% if is_incremental() %}
    WHERE NOT EXISTS (
        SELECT 1 FROM existing e
        WHERE e.hk_<hub> = src.hk_<hub>
        AND e.dss_start_date = src.dss_start_date
    )
    {% endif %}
)
```

```
SELECT
  *,
  'Y' AS dss_is_active,
  CAST(NULL AS DATETIME2) AS dss_end_date
FROM new_records
```

5.6 Dependent Child Satellite (DC Sat) erstellen

Ein **Dependent Child Satellite** wird verwendet, wenn ein Link zusätzliche Schlüsselspalten (DCK - Dependent Child Keys) benötigt, um Zeilen auf einer feineren Granularität zu unterscheiden.

Anwendungsfälle: - Bestellpositionen (Order Product + Positionsnummer) - Kontaktadressen (Person AddressType + Adresse) - Telefonnummern (Company PhoneType + Nummer)

Staging-View Anforderungen Die Staging-View muss für DC Satellites **zusätzliche Hash-Berechnungen** enthalten. Mit `automate_dv.stage()` werden alle Hashes im YAML-Metadata Block definiert:

-- Beispiel: *jira_order_item.sql* (mit DCK: *line_item_no*)

```
{%- set yaml_metadata -%}
source_model:
  staging: "ext_jira_order_item"

derived_columns:
  dss_record_source: "!jira"
  dss_load_date: "COALESCE(TRY_CAST(dss_load_date AS DATETIME2), GETDATE())"
  dss_create_datetime: "GETDATE()"
  dss_business_key: "CONCAT_WS('||', 'default', 'default', ISNULL(LTRIM(RTRIM(CAST(

hashed_columns:
  hk_order_item: "order_id"
  hk_link_order_product:
    - "order_id"
    - "product_id"
    - "line_item_no"
  hd_order_item:
  is_hashdiff: true
  columns:
    - "discount"
    - "quantity"
    - "unit_price"
  hd_order_product_dc:
  is_hashdiff: true
  columns:
```



```

- "discount"
- "line_item_no"
- "quantity"
- "unit_price"
{%- endset -%}

{% set metadata_dict = fromyaml(yaml_metadata) %}

{{ automate_dv.stage(include_source_columns=true,
  source_model=metadata_dict['source_model'],
  derived_columns=metadata_dict['derived_columns'],
  hashed_columns=metadata_dict['hashed_columns']) }}

```

DC Satellite Model

```

-- sat_order_product_dc.sql
{{ config(
  materialized='incremental',
  as_columnstore=false
) }}

{% set yaml_metadata -%}
source_model: "jira_order_item"
src_pk: "hk_link_order_product"
src_hashdiff:
  source_column: "hd_order_product_dc"
  alias: "HASHDIFF"
src_payload:
- "line_item_no"
- "quantity"
- "unit_price"
- "discount"
src_ldts: "dss_load_date"
src_source: "dss_record_source"
src_extra_columns:
- "dss_create_datetime"
{%- endset -%}

{% set metadata_dict = fromyaml(yaml_metadata) %}

{{ automate_dv.sat(
  src_pk=metadata_dict["src_pk"],
  src_hashdiff=metadata_dict["src_hashdiff"],
  src_payload=metadata_dict["src_payload"],
  src_ldts=metadata_dict["src_ldts"],
  src_source=metadata_dict["src_source"],
  src_extra_columns=metadata_dict["src_extra_columns"],
  source_model=metadata_dict["source_model"]

```

```
) }}
```

5.7 Multi-Active Satellite (MA Sat) erstellen

Ein **Multi-Active Satellite** erlaubt **mehrere gleichzeitig gültige Werte** für denselben Business Key.

VS Code Extension: Im Entity Designer Spalten als Multi-Active Key markieren. MA Satellites werden automatisch beim Klick auf "Generate Satellites" oder "Generate All" erstellt. **Wichtig:** MA Sat benötigt mindestens eine Hub-Spalte (Business Key) - im Gegensatz zu DC Sat.

Anwendungsfälle: - Mehrere Telefonnummern pro Kunde (phone_type unterscheidet)
- Mehrere Rollen pro Mitarbeiter (role unterscheidet) - Mehrere Adressen pro Person (address_type unterscheidet)

Staging-View Anforderungen

-- Beispiel: *jira_employee_phone.sql* (mit CDK: *phone_type*)

```
{%- set yaml_metadata -%}
source_model:
  staging: "ext_jira_employee_phone"

derived_columns:
  dss_record_source: "!jira"
  dss_load_date: "COALESCE(TRY_CAST(dss_load_date AS DATETIME2), GETDATE())"
  dss_create_datetime: "GETDATE()"
  dss_business_key: "CONCAT_WS('||', 'default', 'default', ISNULL(LTRIM(RTRIM(CAST(e

hashed_columns:
  hk_employee: "employee_id"
  hd_employee:
    is_hashdiff: true
  columns:
    - "is_primary"
    - "phone_number"
  hd_employee_ma:
    is_hashdiff: true
  columns:
    - "is_primary"
    - "phone_number"
    - "phone_type"
{%- endset -%}

{%- set metadata_dict = fromyaml(yaml_metadata) %}
```

```
{{ automate_dv.stage(include_source_columns=true,
  source_model=metadata_dict['source_model'],
  derived_columns=metadata_dict['derived_columns'],
  hashed_columns=metadata_dict['hashed_columns']) }}
```

MA Satellite Model

```
-- sat_employee_ma.sql
{{ config(
  materialized='incremental',
  as_columnstore=false
) }}

{% set yaml_metadata -%}
source_model: "jira_employee_phone"
src_pk: "hk_employee"
src_cdk:
  - "phone_type"
src_hashdiff:
  source_column: "hd_employee_ma"
  alias: "HASHDIFF"
src_payload:
  - "phone_number"
  - "is_primary"
src_eff: "dss_load_date"
src_ldts: "dss_load_date"
src_source: "dss_record_source"
{% endset -%}

{% set metadata_dict = fromyaml(yaml_metadata) %}

{{ automate_dv.ma_sat(
  src_pk=metadata_dict["src_pk"],
  src_cdk=metadata_dict["src_cdk"],
  src_hashdiff=metadata_dict["src_hashdiff"],
  src_payload=metadata_dict["src_payload"],
  src_eff=metadata_dict["src_eff"],
  src_ldts=metadata_dict["src_ldts"],
  src_source=metadata_dict["src_source"],
  source_model=metadata_dict["source_model"]
) }}
```

Unterschied DC Sat vs MA Sat

Aspekt	DC Sat	MA Sat
Parent	Link	Hub
Hash Key	Link Hash + DCK	Hub Hash

Aspekt	DC Sat	MA Sat
Uniqueness	Link + DCK + Zeit	Hub + CDK + Zeit
automate_dv Macro	sat	ma_sat
Anwendung	Zeilen auf Link-Ebene	Mehrere Werte pro Entity

5.8 PIT Table erstellen

Vorlage: models/business_vault/pit_company.sql

PIT (Point-in-Time) Tabellen für effiziente historische Abfragen.

```
{{ config(
    materialized='table',
    as_columnstore=false
) }}
```

```
WITH snapshot_dates AS (
    SELECT DISTINCT CAST(dss_load_date AS DATE) AS snapshot_date
    FROM {{ ref('sat_<entity>') }}
),
```

```
<entities> AS (
    SELECT DISTINCT hk_<entity>
    FROM {{ ref('hub_<entity>') }}
),
```

```
pit_base AS (
    SELECT
        e.hk_<entity>,
        sd.snapshot_date
    FROM <entities> e
    CROSS JOIN snapshot_dates sd
),
```

```
sat_lookup AS (
    SELECT
        pb.hk_<entity>,
        pb.snapshot_date,
        (
            SELECT TOP 1 s.hk_<entity>
            FROM {{ ref('sat_<entity>') }} s
            WHERE s.hk_<entity> = pb.hk_<entity>
            AND CAST(s.dss_load_date AS DATE) <= pb.snapshot_date
            ORDER BY s.dss_load_date DESC
        ) AS sat_<entity>_hk,
    (
```

```

SELECT TOP 1 s.dss_load_date
FROM {{ ref('sat_<entity>') }} s
WHERE s.hk_<entity> = pb.hk_<entity>
AND CAST(s.dss_load_date AS DATE) <= pb.snapshot_date
ORDER BY s.dss_load_date DESC
) AS sat_<entity>_ldts
FROM pit_base pb
)

SELECT * FROM sat_lookup
WHERE sat_<entity>_hk IS NOT NULL

```

5.9 Current View erstellen (sat*_current_v)

Für jeden Satellite wird eine Current View erstellt, die den aktuellen Stand bereitstellt. Mart-Modelle referenzieren diese Views statt der Satellites direkt.

Datei: models/raw_vault/_common/satellites/sat_<entity>_current_v.sql

```

{{ config(materialized='view') }}
{{ satellite_current_view(
    satellite_model='sat_<entity>',
    hashkey_column='hk_<entity>'
) }}

```

Zugriffsmuster: - **SCD1 (aktueller Stand):** WHERE dss_is_current = 'Y' - **SCD2 (volle Historie):** Kein Filter, alle Records

Datenfluss:

Hub/Sat (vault.*) Current View (sat*_current_v) Mart (mart_*.*)

Mart — Dimensionale Modellierung

Überblick

Der Mart Layer implementiert **Star Schema** (Kimball) als Views auf den Raw Vault. Dimensionen und Faktentabellen verwenden deterministische BIGINT Surrogate Keys.

Datenfluss:

Hub/Sat/Link (vault.*) Current View (sat*_current_v) Dimension/Fakt (mart_<concept>)

Surrogate Key Macro

Alle Dimension Keys verwenden das surrogate_key() Macro (macros/surrogate_key.sql):

```

{{ surrogate_key('business_key_column') }} AS person_key
-- Generiert: ABS(CONVERT(BIGINT, HASHBYTES('MD5', CAST(column AS NVARCHAR(MAX)))))

```

- **Typ:** BIGINT (8 Byte, deterministisch, view-kompatibel, Power BI freundlich)
- **Verwendung:** Dimension PK und Fakt FK (gleicher Aufruf für Join-Kompatibilität)

Dimension Pflicht-Spalten

Spalte	Typ	Beschreibung	Fallback
{dim}_key	BIGINT	Surrogate Key via surrogate_key()	—
{dim}_id	NVARCHAR(255)	Technische ID aus Quellsystem	—
{dim}_code	NVARCHAR(255)	Sprechender Business-Schlüssel	= ID
{dim}_name	NVARCHAR(255)	Bekannte Bezeichnung	= CODE oder 'UNKN
dss_load_date	DATETIME2	Ladezeitpunkt	—
dss_record_source	NVARCHAR(255)	Quellenidentifikation	—

Beispiel: Dimension

Datei: models/mart/project/dim_person.sql

```
{{ config(materialized='view', tags=['dimension']) }}
```

SELECT

```
{{ surrogate_key('lohnnr') }} AS person_key,
CAST(lohnnr AS NVARCHAR(255)) AS person_id,
ISNULL(nachname + ' ' + vorname,
CAST(lohnnr AS NVARCHAR(255))) AS person_code,
ISNULL(nachname + ' ' + vorname, 'UNKNOWN') AS person_name,
sat.dss_load_date,
sat.dss_record_source
FROM {{ ref('hub_person') }} hub
INNER JOIN {{ ref('sat_person__abacus') }} sat
ON hub.hk_person = sat.hk_person
AND sat.dss_is_current = 'Y'
```

Beispiel: Faktentabelle

Datei: models/mart/project/fakt_stunden.sql

```
{{ config(materialized='view', tags=['fact']) }}
```

SELECT

```
{{ surrogate_key('hp.projnr') }} AS projekt_key,
{{ surrogate_key('hpsk.code') }} AS leistungsart_key,
CONVERT(INT, FORMAT(sat.datum, 'yyyyMMdd')) AS datum_key,
sat.azbetint AS stunden,
sat.dss_load_date,
sat.dss_record_source
FROM {{ ref('hub_stundenbuchung') }} h
INNER JOIN {{ ref('sat_stundenbuchung') }} sat
```

```

ON h.hk_stundenbuchung = sat.hk_stundenbuchung
AND sat.dss_is_current = 'Y'
-- ... weitere Joins zu Hubs für FK-Auflösung

```

Naming

Objekt	Pattern	Beispiel
Dimension	dim_{entity}	dim_person, dim_projekt
Faktentabelle	fakt_{content}	fakt_stunden
Schema (domain)	mart_{concept}	mart_project

Materialisierung

- materialized='view' — Standard (Virtualisierung bevorzugt)
- materialized='table' — Nur bei Performance-Bedarf

Schema YAML

Jedes Mart-Modell muss in `__<concept>__models.yml` dokumentiert sein:

```

models:
- name: dim_person
  description: "Dimension: Mitarbeiterstamm"
  config:
  tags: ['dimension']
  columns:
  - name: person_key
    data_type: bigint
    tests: [not_null, unique]
  - name: person_code
    tests: [not_null]
  - name: person_name
    tests: [not_null]

```

ER-Diagramm

Pro Mart-Domain: `design/mart/er-mart-<concept>.mmd`

Agent

@mart-architect — Erstellt Dimensionen und Faktentabellen aus dem Raw Vault

Mart View erstellen (Legacy Pattern)

Hinweis: Dieses Pattern wird durch das obige Star Schema Pattern abgelöst.
Für neue Mart-Objekte verwende Dimensionen und Faktentabellen mit surrogate_key().

Szenario

Eine flache View für BI-Tools (Power BI, Excel) erstellen.

Neue Datei: models/mart/v_<name>.sql

Beispiel: Aktuelle Firmendaten

```
/*  
 * Mart View: v_company_current  
 * Schema: mart_project  
 *  
 * Flache View mit aktuellen Firmendaten für Reporting.  
 */
```

```
{{ config(  
    materialized='view'  
) }}
```

SELECT

```
-- IDs (für Joins)  
h.hk_company,  
h.object_id,  
h.source_table,  
  
-- Stammdaten  
s.name AS company_name,  
s.street,  
s.citycode AS zip_code,  
s.city,  
s.country AS country_id,  
co.name AS country_name,  
  
-- Kontakt  
s.email,  
s.phone,  
s.mobile,  
s.website,  
  
-- Finanzen  
s.iban,  
s.bic,  
s.credit_rating,
```



```

-- Rolle
lr.role_code,
r.role_name,

-- Metadata
s.dss_load_date AS last_updated,
s.dss_record_source AS source_system

FROM {{ ref('hub_company') }} h

-- Aktuelle Satellite-Daten
INNER JOIN {{ ref('sat_company') }} s
  ON h.hk_company = s.hk_company
  AND s.dss_is_current = 'Y'

-- Rolle
LEFT JOIN {{ ref('link_company_role') }} lr
  ON h.hk_company = lr.hk_company
LEFT JOIN {{ ref('ref_role') }} r
  ON lr.role_code = r.role_code

-- Land
LEFT JOIN {{ ref('link_company_country') }} lc
  ON h.hk_company = lc.hk_company
LEFT JOIN {{ ref('sat_country') }} co
  ON lc.hk_country = co.hk_country
  AND co.dss_is_current = 'Y'

-- Ghost Records ausschließen
WHERE h.object_id > 0

```

Konfiguration in dbt_project.yml:

```

models:
  datavault:
    mart:
      +schema: mart_project
      +materialized: view

```

Deployment:

```
dbt run --select v_company_current
```

Static Tables (Persistierte Marts)

Übersicht

Static Tables sind persistierte Tabellen im Mart-Layer mit folgenden Eigenschaften:

Eigenschaft	Beschreibung
Schema	mart_static
Materialisierung	incremental mit MERGE-Strategie
Index	Non-Clustered auf Hash Key
Change Detection	via last_updated (MAX aus Satellite Load Dates)
Tag	static für Batch-Updates

Wann Static Table vs. View?

Kriterium	View	Static Table
Daten-Aktualität	Real-time	Refresh bei dbt run
Komplexität	1-2 JOINS	Viele JOINS
Abfrage-Häufigkeit	Selten	Häufig (BI-Dashboards)
Performance	Gut	Sehr gut (Index)
Speicher	Kein	Tabelle + Index

Static Table erstellen

Neue Datei: models/mart/tables/<name>.sql

Beispiel: Aktuelle Firmendaten (persistiert)

```

/*
 * Static Table: company_current
 * Schema: mart_static
 *
 * Persistierte Tabelle mit aktuellen Firmendaten.
 * Inkrementelle Updates via MERGE auf hk_company.
 */

{{ config(
    materialized='incremental',
    unique_key='hk_company',
    incremental_strategy='merge',
    merge_update_columns=['name', 'city', 'street', 'last_updated'],
    as_columnstore=false,
    tags=['static'],
    post_hook=[
        "{{ create_hash_index('hk_company') }}"
    ]
) }}

WITH source_data AS (
    SELECT
        -- Hash Key
        h.hk_company,

```

```

h.object_id,

-- Payload
s.name,
s.city,
s.street,

-- Metadata
h.dss_load_date AS hub_load_date,
s.dss_load_date AS last_updated

FROM {{ ref('hub_company') }} h

-- Aktuelle Satellite-Daten
LEFT JOIN {{ ref('sat_company') }} s
ON h.hk_company = s.hk_company
AND s.dss_is_current = 'Y'

-- Ghost Records ausschließen
WHERE h.object_id > 0
)

SELECT * FROM source_data
{% if is_incremental() %}
WHERE last_updated > (SELECT MAX(last_updated) FROM {{ this }})
{% endif %}

```

Index Macros

Die folgenden Macros stehen für Index-Erstellung zur Verfügung:

Macro	Beschreibung
<code>create_hash_index(column)</code>	Non-Clustered Index auf einer Spalte
<code>create_composite_index([col1, col2])</code>	Index auf mehreren Spalten
<code>drop_and_create_index(column)</code>	Index neu erstellen (bei Schema-Änderungen)

Verwendung als post_hook:

```

{{ config(
  post_hook=[
    "{{ create_hash_index('hk_company') }}",
    "{{ create_composite_index(['hk_company', 'hk_project']) }}"
  ]
) }}

```

Deployment

Initial Load (Full Refresh) - ERSTE AUSFÜHRUNG

```
dbt run --select company_current --full-refresh
```

Inkrementelles Update

```
dbt run --select company_current
```

Alle Static Tables aktualisieren

```
dbt run --select tag:static
```

Full Refresh für alle Static Tables (z.B. wöchentlich)

```
dbt run --select tag:static --full-refresh
```

Index verifizieren

-- Index prüfen

SELECT

```
  i.name AS index_name,  
  i.type_desc,  
  c.name AS column_name
```

FROM sys.indexes i

JOIN sys.index_columns ic **ON** i.object_id = ic.object_id **AND** i.index_id = ic.index_id

JOIN sys.columns c **ON** ic.object_id = c.object_id **AND** ic.column_id = c.column_id

WHERE i.object_id = OBJECT_ID('mart_static.company_current')

CLI Command

Mit Agent

```
/create-static-table company_current
```

Mit MCP Tool

```
Tool: create_static_table
```

```
Args: {
```

```
  "tableName": "company_current",
```

```
  "description": "Aktuelle Firmendaten für BI",
```

```
  "baseHub": "hub_company",
```

```
  "satellites": [{ "name": "sat_company", "columns": ["*"], "currentOnly": true }]
```

```
}
```

Security in Mart-Models (RLS/CLS)

Architektur & Begründungen: [datavault-security-architecture.md](#) · DB-

Rollout: [security/DEPLOYMENT.md](#)

Security wird **an der Mart-Grenze** durchgesetzt. Für Modellentwickler gelten vier Regeln:

Regel 1: RLS-Schlüssel in Fakt-Models führen

Jedes RLS-pflichtige Fakt-Model bekommt eine Spalte `dss_sec_value_key` (Format 'ewb' bzw. 'ewb|<kontextwert>', Mandant kommt automatisch aus dem dbt-Target):

```
-- in der SELECT-Liste (Beispiel Finance: Kostenstelle als Kontext):  
{{ sec_value_key('CAST(TRY_CAST(b.kst AS INT) AS NVARCHAR(50))') }} AS dss_sec_val
```

Regel 2: Physische Mart-Tabelle Security Policy via Hook-Paar

Physische Tabellen (`materialized='table'`) sind durch den Schema-Grant direkt lesbar und brauchen eine native Security Policy. **Hooks immer paarweise** — nie einzeln entfernen (Tabelle mit Policy kann nicht gedroppt werden jeder `dbt run` schlägt fehl):

```
{{ config(  
    materialized='table',  
    pre_hook=["{{ drop_security_policy() }}"],  
    post_hook=["{{ apply_security_policy('finance') }}"]  
) }}
```

Regel 3: Publierte _v-Views rls_filter

```
SELECT *  
FROM {{ ref('fakt_buchungen') }}  
WHERE {{ rls_filter('finance') }}
```

Regel 4: PII-Spalten cls_mask, Tier-1 nie in den Mart

```
-- Tier 2 (PII): maskieren, Kontext 'person_pii'  
{{ cls_mask('person_name', 'person_pii') }} AS person_name,  
{{ cls_mask('geburtsdatum', 'person_pii', 'NULL') }} AS geburtsdatum,
```

Tier-1-Spalten (SOC_INSURANCE_NR, ZEMIS_NR, BADGE_ID) dürfen in **keinem** Mart-Objekt auftauchen — Vorsicht bei `SELECT *` aus Satellites! Der Singular-Test `tests/security/assert_no_tier1_columns_in_mart.sql` schlägt sonst an.

Referenz-Implementierungen

Muster	Model
Fakt-Tabelle mit Policy-Hooks + <code>dss_sec_value_key</code>	<code>models/mart/finance/fakt_buchungen.sql</code>
_v-View mit <code>rls_filter</code>	<code>models/mart/finance/fakt_buchungen_v.sql</code>
Dimension mit <code>cls_mask</code>	<code>models/mart/project/dim_person_v.sql</code>

Häufige Fehler

Symptom	Ursache
dbt run: „Invalid object name 'sec.fn_check_rls'“ dbt run: Tabelle kann nicht gedroppt werden Marts/Tests plötzlich leer, kein Fehler	security/ddl/*.sql auf der Ziel-DB nicht deployed (siehe DEPLOYMENT.md) pre_hook mit drop_security_policy() fehlt dbt-Service-User-Exemption (no_sec=1) fehlt — Test assert_dbt_service_user_exemption prüft das
Neue Objekte für User nicht sichtbar	Kein Problem — Schema-Grants decken neue Objekte automatisch ab; nur neue Schemas brauchen ein neues OLS-Skript

Tests hinzufügen

Test-Typen

Test	Zweck	Beispiel
not_null	Spalte darf nicht NULL sein	Primary Keys, Business Keys
unique	Werte müssen eindeutig sein	Hash Keys in Hubs
relationships	FK-Beziehung validieren	Satellite Hub
accepted_values	Nur bestimmte Werte erlaubt	Status-Felder

Tests in schema.yml

Datei: models/schema.yml

```
models:
- name: hub_<entity>
  columns:
- name: hk_<entity>
  tests:
- unique
- not_null
- name: <business_key>
  tests:
- not_null

- name: sat_<entity>
  columns:
- name: hk_<entity>
  tests:
- not_null
```

```

- relationships:
  to: ref('hub_<entity>')
  field: hk_<entity>
- name: hd_<entity>
tests:
- not_null
- name: dss_is_current
tests:
- accepted_values:
values: ['Y', 'N']

```

Tests ausführen

Alle Tests

```
dbt test
```

Tests für bestimmtes Model

```
dbt test --select hub_company
```

Tests für Tag

```
dbt test --select tag:hub
```

Deployment Workflow

GitHub Actions CI/CD Pipeline

Das Projekt verwendet **GitHub Actions** für automatisiertes Deployment. Der Self-hosted Runner läuft auf der gleichen VM wie die Entwicklungsumgebung.

Verfügbare Workflows

Workflow	Trigger	Zweck
CI	PR nach main/dev	Validierung (compile + test)
Deploy Dev	Push auf main / manual	Deployment nach Vault (Dev)
Deploy Prod	Tag v* / manual + Approval	Deployment nach Vault_Jira
Docs	Push auf main / manual	dbt docs GitHub Pages

Workflow manuell ausführen

Deploy Dev manuell triggern

```
gh workflow run deploy-dev.yml --ref main
```

Deploy Prod manuell triggern (erfordert Approval!)

```
gh workflow run deploy-prod.yml --ref main -f target=jira
```

```
# Docs generieren
gh workflow run docs.yml --ref main
```

Workflow-Status prüfen

```
# Letzte Runs anzeigen
gh run list --limit 5

# Bestimmten Run beobachten
gh run watch <run-id>

# Logs eines fehlgeschlagenen Runs
gh run view <run-id> --log-failed
```

Manuelles Deployment (Lokal)

Falls die Pipeline nicht verwendet werden soll:

```
#
# DEVELOPMENT ||
#

# 1. Änderungen entwickeln
dbt run --select <changed_models>

# 2. Tests lokal ausführen
dbt test --select <changed_models>

# 3. SQL prüfen
dbt compile --select <model>
cat target/compiled/datavault/models/path/to/model.sql

# 4. Git Commit & Push
git add .
git commit -m "feat: Add <feature>"
git push origin dev

# 5. Pull Request erstellen CI läuft automatisch
gh pr create --base main --head dev --title "feat: <feature>"

#
# PRODUCTION (via CI/CD) ||
#

# 6. PR mergen Deploy Dev läuft automatisch
gh pr merge <pr-number> --squash

# 7. Für Prod: Tag erstellen oder manuell triggern
```



```
git tag v1.0.0 && git push origin v1.0.0
# ODER
gh workflow run deploy-prod.yml --ref main -f target=jira
# Approval in GitHub erforderlich!
```

Manuelles Prod-Deployment (ohne CI/CD)

```
#
# PRODUCTION (manuell) ||
#

# 1. External Tables in Prod erstellen/aktualisieren
dbt run-operation stage_external_sources --target jira

# 2. Seeds laden (falls geändert)
dbt seed --target jira

# 3. Models deployen
dbt run --target jira

# 4. Tests in Prod
dbt test --target jira
```

Full Refresh (Schema-Änderungen)

```
# Bei Spaltenänderungen: Full Refresh erforderlich!
dbt run --full-refresh --target jira
```

Troubleshooting

Häufige Fehler

Fehler	Ursache	Lösung
Column not found	Spalte fehlt in External Table	sources.yml prüfen, stage_external_sources ausführen
Columnstore not supported	Azure SQL Basic Tier	+as_columnstore: false in Config
Hash Diff changed unexpectedly	Neue Spalte ohne Full Refresh	dbt run --full-refresh
Duplicate key	Unique-Constraint verletzt	Hash Key Berechnung prüfen
Cross-database reference	Hardcoded Database	{{ target.database }} verwenden
Login timeout	Azure Token abgelaufen	az login ausführen

Debug-Befehle

```
# Generiertes SQL anzeigen
dbt compile --select <model>
cat target/compiled/datavault/models/path/to/model.sql

# Logs prüfen
less logs/dbt.log

# Letzte Query
cat target/run/datavault/models/path/to/model.sql

# Verbindung testen
dbt debug
```

Checklisten

Neue Entity Checkliste

- ☐ External Table in sources.yml definiert
- ☐ Staging View erstellt (<concept>_<entity>.sql)
 - ☐ automate_dv.stage() YAML Metadata Pattern
 - ☐ Hash Key definiert in hashed_columns
 - ☐ Hash Diff definiert (falls Satellite)
 - ☐ Reserved Keywords via _escape escaped
 - ☐ Metadata-Spalten als derived_columns
- ☐ Hub erstellt (hub_<entity>.sql)
- ☐ Satellite erstellt (sat_<entity>.sql)
 - ☐ Post-Hook für dss_is_current
- ☐ Link erstellt (falls Beziehung)
- ☐ Tests in schema.yml hinzugefügt
- ☐ dbt run-operation stage_external_sources
- ☐ dbt run --select +raw_vault.<concept>.hub_<entity> +raw_vault.<concept>.sat_<entity>
- ☐ dbt test --select raw_vault.<concept>
- ☐ Ghost Records erweitert (optional)
- ☐ Dokumentation aktualisiert

Attribut hinzufügen Checkliste

- ☐ Spalte in sources.yml hinzugefügt
- ☐ Spalte in Staging View hashed_columns/hashdiff hinzugefügt (falls getrackt)
- ☐ Spalte in Satellite src_payload hinzugefügt
- ☐ dbt run-operation stage_external_sources
- ☐ dbt run --full-refresh --select <concept>_* sat_*
- ☐ dbt test

Pre-Deployment Checkliste

- ☐ Alle Tests lokal bestanden
- ☐ SQL kompiliert und geprüft
- ☐ Keine hardcoded Datenbanknamen
- ☐ +as_columnstore: false gesetzt
- ☐ Hash-Separator ist '||' (concat_string in dbt_project.yml)
- ☐ Git committed und gepusht
- ☐ PR erstellt und CI erfolgreich

CI/CD Troubleshooting

Problem	Lösung
CI läuft nicht	Prüfen ob Änderungen in models/, macros/, etc. (Path Filter!)
Profile not found	profile: in dbt_project.yml muss mit profiles.yml übereinstimmen
Runner offline	sudo systemctl restart actions.runner.fellnerd-datavault-dbt.dbt-runner-vm
Prod-Tests fehlen	dbt seed --target jira ausführen
Azure Login failed	Service Principal Secret ggf. abgelaufen, neu generieren

Weiterführende Dokumentation

Dokument	Inhalt	Link
System-Dokumentation	Architektur, Komponenten	SYSTEM.md
User-Dokumentation	Endanwender-Guide	USER.md
Model Architecture	Datenmodell, ERD	MODEL_ARCHITECTURE.md
Security-Architektur	OLS/RLS/CLS/CLE, Mandantenmodell	datavault-security-architecture.md
Security-Deployment	Runbook für DB-Rollout	security/DEPLOYMENT.md
Lessons Learned	Entscheidungen, Troubleshooting	LESSONS_LEARNED.md
Copilot Instructions	KI-Assistenz Regeln	copilot-instructions.md
CI/CD Plan	Pipeline-Implementierung	plan-githubActionsCiCd.prompt.prompt.
dbt Docs	Generierte Dokumentation	fellnerd.github.io/datavault-dbt

Dokument	Inhalt	Link
GitHub Actions	Pipeline-Runs	Actions

Letzte Aktualisierung: 2025-12-27